



Green University of Bangladesh

*Department of Computer Science and Engineering (CSE)
Semester: (Fall, Year: 2025), B.Sc. in CSE (Day)*

SentinelShield: Advanced Linux Security Hardening and Threat Audit Framework

*Course Title: Operating Systems Lab
Course Code: CSE-402
Section: 231-D2*

Students Details

Name	ID
Promod Chandra Das	231002005
Chinmoy Debnath	231902029

*Submission Date: 28/12/2025
Course Teacher's Name: Md. Shoaib Alam*

[For teachers use only: **Don't write anything inside this box**]

<u>Lab Project Status</u>	
Marks:	Signature:
Comments:	Date:

Contents

1	Introduction	2
1.1	Overview	2
1.2	Motivation	2
1.3	Problem Definition	2
1.3.1	Problem Statement	2
1.3.2	Complex Engineering Problem	3
1.4	Design Goals/Objectives	3
1.5	Application	3
2	Design/Development/Implementation of the Project	5
2.1	Introduction	5
2.2	Implementation	5
2.2.1	Implementation and Workflow	5
2.3	Algorithms	15
3	Performance Evaluation	17
3.1	Simulation Environment/ Simulation Procedure	17
3.2	Results Analysis/Testing	19
3.3	Results Overall Discussion	26
3.3.1	Complex Engineering Problem Discussion	26
4	Conclusion	28
4.1	Discussion	28
4.2	Limitations	28
4.3	Scope of Future Work	29

Chapter 1

Introduction

1.1 Overview

SentinelShield is a Bash-based Linux security auditing and hardening framework designed to automatically detect vulnerabilities and strengthen system security. It scans the system for weak configurations, insecure permissions, exposed ports, unnecessary services, and potential attack vectors. Using a policy-driven approach, it evaluates the system against predefined security standards, applies corrective hardening actions, and generates detailed security reports. SentinelShield also maintains backups and supports rollback to ensure safe and reliable system modifications..

1.2 Motivation

Linux systems are widely used in servers, cloud platforms, and enterprise infrastructures. However, most security breaches occur due to simple misconfigurations, weak SSH settings, and poorly managed permissions. Manual security auditing is slow, inconsistent, and prone to human error. Many organizations lack lightweight and customizable tools that can automatically assess and secure Linux environments. SentinelShield is motivated by the need for a simple, fast, and automated framework that ensures consistent system security and reduces administrative workload [1].

1.3 Problem Definition

1.3.1 Problem Statement

Modern Linux environments contain numerous security risks arising from improper configurations and overlooked vulnerabilities. Issues such as world-writable files, open ports, SUID/SGID binaries, insecure SSH settings, and unnecessary services often remain undetected. Administrators struggle to manually maintain security across multiple systems due to complexity and time constraints.

1.3.2 Complex Engineering Problem

The following table must be completed according to your above discussion in detail. The column on the right side should be filled only on the attributes you have chosen to be touched by your own project.

1.4 Design Goals/Objectives

The primary objectives of SentinelShield are: 1. **Automated Vulnerability Scanning:** Detect insecure permissions, misconfigured services, SSH weaknesses, and exposed attack surfaces. 2. **Policy-Based Validation:** Use a customizable INI-style security policy to define standards for system configuration. 3. **Automated System Hardening:** Apply safe fixes by adjusting permissions, modifying SSH settings, enabling unnecessary services, and securing system resources. 4. **Backup and Rollback Features:** Store backups of all modified configurations and support reverting to previous states. 5. **Detailed Security Reporting:** Generate structured reports including identified risks, actions taken, and an overall security score. 6. **Lightweight and Portable Implementation:** Build the entire system using pure Bash to ensure compatibility and low resource usage

1.5 Application

SentinelShield can be applied in multiple practical contexts across both academic and real-world environments. In enterprise server security, it can be used to audit and automatically harden production Linux servers, reducing the risk of misconfigurations and unauthorized access. In DevOps and system administration, SentinelShield can be integrated into deployment or server-setup workflows to enforce consistent security policies during system initialization. For cybersecurity training and education, the framework provides learners with a hands-on tool to understand Linux security mechanisms, policy validation, and safe configuration management. In cloud infrastructure protection, SentinelShield can be used to assess and secure virtual machines deployed on platforms such as AWS, Azure, and Google Cloud. Additionally, for personal security use, it enables individual users to strengthen their Linux systems by identifying security weaknesses and applying controlled hardening measures to protect against unauthorized access..

In every section please add subsections and figures and citations as references [1] also.

Table 1.1: Summary of the attributes touched by the mentioned projects

Name of the P Attributes	Explain how to address
P1: Depth of knowledge required	The project requires strong understanding of Linux internals, file permissions, process management, SSH security, network services, and Bash scripting. It demands advanced knowledge of cybersecurity concepts and OS-level hardening mechanisms.
P2: Range of conflicting requirements	The tool must secure the system without disrupting essential services, balance automation with user safety, maintain compatibility across distributions, and enforce strict security while preserving system usability.
P3: Depth of analysis required	The project performs multi-layer analysis of permissions, services, ports, cronjobs, configurations, and SSH rules. It evaluates each item against policy standards, requiring deep technical inspection and logical decision-making.
P4: Familiarity of issues	The project deals with real-world security issues such as misconfigured SSH, world writable files, insecure services, privilege escalation paths, and unnecessary running daemons that commonly exist in Linux systems.
P5: Extent of applicable codes	The project applies Linux hardening best practices, Bash automation techniques, security benchmarks (CIS-like standards), permission models, and safe configuration editing methods with backup and rollback mechanisms.
P6: Extent of stakeholder involvement and conflicting requirements	Different stakeholders (administrators, DevOps teams, security officers, and end-users) have varying needs such as maximum security, high uptime, ease of maintenance, and system stability. The project balances these through customizable security policies.
P7: Interdependence	The system interacts with multiple Linux subsystems such as filesystems, SSH, services, processes, and network configurations.

Chapter 2

Design/Development/Implementation of the Project

2.1 Introduction

Linux operating systems are widely used in modern computing environments due to their stability and flexibility, but improper configuration and unsecured services can expose systems to significant security risks. Regular security auditing and controlled hardening are therefore essential to maintain a secure and reliable system.

SentinelShield is a Bash-based Linux security audit and hardening framework designed to analyze system security, validate configurations against predefined policies, and safely apply SSH hardening using backup, validation, and rollback mechanisms. The framework provides a menu-driven, user-friendly interface that allows users to perform security checks and view results easily in a terminal environment.

In addition to security auditing, SentinelShield includes CPU scheduling simulations and process priority demonstrations to support operating system laboratory objectives. By combining security assessment, policy-based decision-making, and OS-level concepts in a single framework, the project offers a practical and educational approach to understanding Linux system administration and security. [2] [3] [4].

2.2 Implementation

2.2.1 Implementation and Workflow

.

The workflow

1. The workflow of SentinelShield follows a structured and secure approach to Linux system assessment and hardening. The framework operates through a sequential process to ensure safety, reliability, and clear decision-making.

```

set -u

BACKUP_DIR="$HOME/sentinel_backup"
TMPDIR="/tmp"
POLICY_FILE="$BACKUP_DIR/sentinel_policy.ini"
LOGFILE="$BACKUP_DIR/sentinel.log"
SSH_CFG="/etc/ssh/sshd_config"

FIND_PRUNE='( -path /proc -o -path /sys -o -path /run -o -path /dev ) -prune -o'

# ----- UI tuning (reduces ugly look) -----
export NEWT_COLORS='
root=white,black
window=white,black
border=white,black
title=green,black
textbox=white,black
button=black,green
compactbutton=white,black
entry=white,black
label=cyan,black
actbutton=black,green
listbox=white,black
actselllistbox=black,cyan
checkbox=green,black
'

need() { command -v "$1" >/dev/null 2>&1 || { echo "Missing dependency: $1"; exit 1; }
have() { command -v "$1" >/dev/null 2>&1; }
is_root() { [ "$(id -u)" -eq 0 ]; }
ensure() { mkdir -p "$BACKUP_DIR"; touch "$LOGFILE" 2>/dev/null || true; }
log(){ ensure; echo "[$(date '+%F %T')] $*" >> "$LOGFILE"; }

# ----- small UI helpers -----
msg() { whiptail --title "$1" --msgbox "$2" 12 86; }
bigmsg() { whiptail --title "$1" --msgbox "$2" 16 92; }
show() { whiptail --title "$1" --scrolltext --textbox "$2" 24 94; }
confirm() { whiptail --title "$1" --yesno "$2" 12 86; }

# ----- Policy -----
ensure_policy() {
    ensure
    [ -f "$POLICY_FILE" ] && return

```

Figure 2.1: initial 26

```

cat >"$POLICY_FILE"<<"EOF"
[SSH]
PermitRootLogin=no
PasswordAuthentication=no

[SCAN]
MaxWorldWritableFiles=50
MaxWorldWritableDirs=50
EOF
log "Created policy $POLICY_FILE"
}

ini_get(){ # section key file
awk -v s="$1" -v k="$2" '
$0!~/[[:space:]]+\\.[^\\][[:space:]]+$/ {in=$0;next}
in ~ $0 ~/[[:space:]]+$/ {
split($0,a,"="); k=a[1]; v=a[2]
gsub(/^[[:space:]]+|[[:space:]]+$/,"",k); gsub(/^[[:space:]]+|[[:space:]]+$/,"",v)
if(k==K){print v; exit}
}
' "$3"
}

# ----- SSH detection -----
ssh_installed() { [ -f "$SSH_CFG" ] || have sshd || [ -x /usr/sbin/sshd ]; }

ssh_service_name() {
systemctl list-unit-files 2>/dev/null | awk '{print $1}' | grep -qx "ssh.service" 66 { echo "ssh"; return; }
systemctl list-unit-files 2>/dev/null | awk '{print $1}' | grep -qx "sshd.service" 66 { echo "sshd"; return; }
systemctl is-active ssh 2>/dev/null >/dev/null 66 { echo "ssh"; return; }
systemctl is-active sshd 2>/dev/null >/dev/null 66 { echo "sshd"; return; }
echo ""
}

ssh_status() {
local s; s=$(ssh_service_name)
[ -n "$s" ] 66 systemctl is-active "$s" 2>/dev/null || echo "NOT FOUND"
}

ssh_restart() {
local s; s=$(ssh_service_name)
[ -n "$s" ] || return 1
systemctl restart "$s" 2>/dev/null
}

sshd_bin() {
have sshd 66 { command -v sshd; return; }

```

Figure 2.2: initial 27

```

[ -x /usr/sbin/sshd ] 66 { echo "/usr/sbin/sshd"; return; }
echo ""
}

sshd_validate() {
local b; b=$(sshd_bin)
[ -n "$b" ] || return 1
"$b" -t 2>/dev/null
}

sshd_get(){ # key → last value
[ -f "$SSH_CFG" ] || { echo ""; return; }
awk -v k="$1" '
$0!~/[[:space:]]+$/ {
END{print v}
' "$SSH_CFG"
}

sshd_set(){ # key value
[ -f "$SSH_CFG" ] || return 1
local k="$1" v="$2" tmp="$TMPDIR/sshd_config.sentinel.$$"
awk -v k="$1" '
{ if($0!~/[[:space:]]+$/){print; next}
if(tolower($1)==k) next
print
' "$SSH_CFG" >"$tmp" || return 1
echo "$k $v" >>"$tmp"
cp "$tmp" "$SSH_CFG" 66 rm -f "$tmp"
}

backup_sshd(){
ensure
[ -f "$SSH_CFG" ] || return 1
local ts; ts=$(date +%F_%H-%M-%S)
cp "$SSH_CFG" "$BACKUP_DIR/sshd_config.$ts.bak" 2>/dev/null || return 1
echo "$BACKUP_DIR/sshd_config.$ts.bak"
}

# ----- status line -----
status_line() {
local r s
r="USER"
is_root 66 r="ROOT"
s="SSH:N/A"

```

Figure 2.3: initial 28


```

ssh_installed || s="SSH:${ssh_status}"
echo "[${r}] $s Policy:${(basename "$POLICY_FILE")} Log:${(basename "$LOGFILE")}"
}

# ----- Intro + dependency check -----
intro() {
    ensure_policy; ensure
    local T="$TMPDIR/sentinel_intro.txt"
    {
        echo "SentinelShield v1.1"
        echo "Advanced Linux Security Hardening & Threat Audit Framework"
        echo
        echo "Purpose: Detect → Validate → Harden → Verify → Report"
        echo
        echo "Modules:"
        echo " • Security Audit (permissions, ports, suid/sgid, services)"
        echo " • Policy-based PASS/FAIL (INI policy)"
        echo " • Safe SSH Hardening (backup + validate + restart + rollback)"
        echo " • Security Report + Score + Logs"
        echo " • CPU Scheduling (FCFS/SJF/RR) + renice demo (OS Lab)"
        echo
        echo "-----"
        echo "${status_line}"
    } >"$T"
    show "SentinelShield" "$T"
}

env_check() {
    local T="$TMPDIR/sentinel_env.txt"
    {
        echo "≡ Environment Check ≡"
        echo "Generated: $(date)"
        echo
        echo "Core:"
        echo " - whiptail: OK"
        echo
        echo "Audit tools:"
        have ss || echo " - ss: OK" || echo " - ss: MISSING (install iproute2)"
        have netstat || echo " - netstat: OK" || echo " - netstat: optional"
        echo
        echo "SSH hardening tools:"
        if ssh_installed; then
            echo " - OpenSSH Server: INSTALLED"
            echo " - sshd_config: $SSH_CFG"
            echo " - service: ${ssh_status}"
        else

```

Figure 2.4: initial 29

```

    echo " - OpenSSH Server: NOT INSTALLED (Hardening/Rollback will be N/A)"
    echo "   Install:"
    echo "     sudo apt install openssh-server"
    echo "     sudo systemctl enable --now ssh"
  fi
  echo "Tip: Run with sudo for full coverage."
} >"$T"
show "Environment Check" "$T"

# ----- Audit -----
audit_basic_scan() {
  local T="$TMPDIR/sentinel_scan.txt"
  {
    echo "=== Basic Security Scan ==="
    echo "Generated: $(date)"
    echo "-----"
    echo "${status_line}"
    echo "[+] World-writable files (top 10, REAL FS only):"
    local wf
    wf=$(eval find / $FIND_PRUNE -type f -perm -0002 -printf '%m %u:%g %p\n' 2>/dev/null | head -n 10)
    [ -n "$wf" ] && echo "$wf" || echo "None found."
    echo "[+] World-writable directories (top 10, REAL FS only):"
    local wd
    wd=$(eval find / $FIND_PRUNE -type d -perm -0002 -printf '%m %u:%g %p\n' 2>/dev/null | head -n 10)
    [ -n "$wd" ] && echo "$wd" || echo "None found."
    echo "[+] Sticky-bit check (expected 1777 with 't'):"
    for d in /tmp /var/tmp /dev/shm /run/lock /run/screen; do
      [ -e "$d" ] && stat -c "%A %a %U:%G %n" "$d" 2>/dev/null || true
    done
  } >"$T"
  log "Audit: basic scan"
  show "Audit - Basic Scan" "$T"
}

audit_ports() {
  local T="$TMPDIR/sentinel_ports.txt"
  {
    echo "=== Open / Listening Ports ==="
    echo "Generated: $(date)"
    echo "-----"

```

Figure 2.5: initial 30

```

    echo "${status_line}"
    echo
    if have ss; then
      ss -tulpen 2>/dev/null | head -n 70
    elif have netstat; then
      netstat -tulpen 2>/dev/null | head -n 70
    else
      echo "No ss/netstat found. Install iproute2 or net-tools."
    fi
  } >"$T"
  log "Audit: ports"
  show "Audit - Ports" "$T"
}

audit_suid() {
  local T="$TMPDIR/sentinel_suid.txt"
  local out
  out=$(eval find / $FIND_PRUNE -type f \( -perm -4000 -o -perm -2000 \) 2>/dev/null | head -n 60)
  {
    echo "=== SUID/SGID Binaries (REAL FS only) ==="
    echo "Generated: $(date)"
    echo "-----"
    echo "${status_line}"
    echo
    [ -n "$out" ] && echo "$out" || echo "None found (or restricted permissions)."
  } >"$T"
  log "Audit: suid/sgid"
  show "Audit - SUID/SGID" "$T"
}

audit_services() {
  local T="$TMPDIR/sentinel_services.txt"
  {
    echo "=== Services Overview ==="
    echo "Generated: $(date)"
    echo "-----"
    echo "${status_line}"
    echo "[+] Running services (top 50):"
    systemctl list-units --type=service --state=running 2>/dev/null | head -n 50 || echo "systemctl not available"
    echo "[+] Enabled services (top 50):"

```

Figure 2.6: initial 31

```

systemctl list-units --type=service --state=running 2>/dev/null | head -n 50 || echo "systemctl not available"
echo
echo "[+] Enabled services (top 50):"
systemctl list-unit-files --type=service --state=enabled 2>/dev/null | head -n 50 || echo "systemctl not available"
} > "$T"
log "Audit: services"
show "Audit - Services" "$T"
}

audit_menu() {
while true; do
local c
c=$(whiptail --title "Security Audit" --menu "${status_line}\n\nSelect: 20 86 10 \
"1" "Basic Scan (permissions + sticky-bit)" \
"2" "Open Ports Scan" \
"3" "SUID/SGID Scan" \
"4" "Services Overview" \
"5" "Back" 3>81 1>82 2>83) || return
case "$c" in
1) audit_basic_scan ;;
2) audit_ports ;;
3) audit_suid ;;
4) audit_services ;;
5) return ;;
*) return ;;
esac
done
}

# ----- Hardening -----
policy_check() {
ensure_policy
local T="$TMPDIR/sentinel_policy.txt"
{
echo "=== Policy Validation (INI) ==="
echo "Policy file: $POLICY_FILE"
echo "Generated: $(date)"
echo "-----"
echo "${status_line}"
echo
if ! ssh installed; then
echo "SSH NOT installed -> SSH checks are N/A"
echo
echo "Install to demo hardening:"
echo "  sudo apt install openssh-server"
echo "  sudo systemctl enable --now ssh"
else
local w_prl w_pa c_prl c_pa

```

Figure 2.7: initial 32

```

w_prl=$(ini_get SSH PermitRootLogin "$POLICY_FILE"); [ -z "$w_prl" ] && w_prl=no
w_pa=$(ini_get SSH PasswordAuthentication "$POLICY_FILE"); [ -z "$w_pa" ] && w_pa=no
c_prl=$(sshd_get PermitRootLogin); [ -z "$c_prl" ] && c_prl="(not set)"
c_pa=$(sshd_get PasswordAuthentication); [ -z "$c_pa" ] && c_pa="(not set)"

echo "Rule: Unset SSH options are NON-COMPLIANT (FAIL)."
```

```

echo "[*] PermitRootLogin"
echo "  Current: $c_prl"
echo "  Policy: $w_prl"
[ $(echo "$c_prl" | tr A-Z a-z) = "$(echo "$w_prl" | tr A-Z a-z)" ] && echo "    => PASS" || echo "    => FAIL"
echo "[*] PasswordAuthentication"
echo "  Current: $c_pa"
echo "  Policy: $w_pa"
[ $(echo "$c_pa" | tr A-Z a-z) = "$(echo "$w_pa" | tr A-Z a-z)" ] && echo "    => PASS" || echo "    => FAIL"
} > "$T"
log "Hardening: policy check"
show "Policy Check" "$T"
}

apply_hardening() {
ensure_policy
is_root || { msg "Hardening" "Run with sudo to apply hardening."; return; }

if ! ssh installed; then
msg "Hardening" "OpenSSH Server NOT installed.\nHardening is N/A.\n\nInstall:\n  sudo apt install openssh-server\n  sudo systemctl enable --now ssh"
return
fi

confirm "Hardening" "Will update SSH config based on policy.\nBackup will be created.\nProceed?" || return

local w_prl w_pa bak
w_prl=$(ini_get SSH PermitRootLogin "$POLICY_FILE"); [ -z "$w_prl" ] && w_prl=no
w_pa=$(ini_get SSH PasswordAuthentication "$POLICY_FILE"); [ -z "$w_pa" ] && w_pa=no
bak=$(backup_sshd) || { msg "Hardening" "Backup failed. Aborting."; return; }

sshd_set PermitRootLogin "$w_prl" || { msg "Hardening" "Failed to set PermitRootLogin"; return; }
sshd_set PasswordAuthentication "$w_pa" || { msg "Hardening" "Failed to set PasswordAuthentication"; return; }

if ssh_validate; then
if ssh_restart; then

```

Figure 2.8: insert 33

```

if sshd_validate; then
    if ssh_restart; then
        log "Hardening applied. Backup=$bak"
        msg "Hardening" "Hardening applied successfully.\nBackup:\n$bak"
    else
        cp "$bak" "$SSH_CFG" 2>/dev/null || true
        ssh_restart 2>/dev/null || true
        log "Hardening restart failed; rolled back. Backup=$bak"
        msg "Hardening" "SSH restart failed.\nRolled back:\n$bak"
    fi
else
    cp "$bak" "$SSH_CFG" 2>/dev/null || true
    log "Hardening invalid config; rolled back. Backup=$bak"
    msg "Hardening" "Validation failed (sshd -t).\nRolled back:\n$bak"
fi

rollback() {
    ensure
    is_root || { msg "Rollback" "Run with sudo to rollback."; return; }

    if ! ssh_installed; then
        msg "Rollback" "OpenSSH Server NOT installed.\nRollback is N/A."
        return
    fi

    local b
    b=$(ls -1 "$BACKUP_DIR"/sshd_config.*.bak 2>/dev/null | sort -r | head -n 15 || true)
    [ -z "$b" ] && { msg "Rollback" "No backups found in:\n$BACKUP_DIR"; return; }

    local items=() i=1 f
    while IFS= read -r f; do items+=("$i" "${basename "$f"}"); i=$((i+1)); done <<"$b"

    local c sel
    c=$(whiptail --title "Rollback" --menu "Select backup to restore:" 22 86 15 "${items[@]}" 3>61 1>62 2>63 || return)
    sel=$(echo "$b" | sed -n "${c}p"); [ -z "$sel" ] && return

    confirm "Rollback" "Restore this backup?\n\n$(basename "$sel")" || return
    cp "$sel" "$SSH_CFG" 2>/dev/null || { msg "Rollback" "Restore failed."; return; }
    sshd_validate && ssh_restart 2>/dev/null || true
    log "Rollback restored $sel"
    msg "Rollback" "Restored:\n$sel"
}

hardening_menu() {
    while true; do

```

Figure 2.9: initial 34

```

local c
c=$(whiptail --title "SSH Hardening" --menu "${status_line}\n\nSelect:" 20 86 10 \
    "1" "Policy PASS/FAIL (INI)" \
    "2" "Apply SSH Hardening (safe)" \
    "3" "Rollback SSH Config" \
    "4" "Back" 3>61 1>62 2>63 || return)
case "$c" in
    1) policy_check ;;
    2) apply_hardening ;;
    3) rollback ;;
    4) return ;;
esac
done

# ----- Reports -----
report_generate() {
    ensure_policy; ensure
    local ts rep score=100
    ts=$(date +%F_%H-%M-%S)
    rep="$BACKUP_DIR/security_report_$ts.txt"

    local maxf maxd wwf wwd
    maxf=$(ini_get SCAN MaxWorldWritableFiles "$POLICY_FILE"); [ -z "$maxf" ] && maxf=50
    maxd=$(ini_get SCAN MaxWorldWritableDirs "$POLICY_FILE"); [ -z "$maxd" ] && maxd=50
    wwf=$(eval find / $FIND_PRUNE -type f -perm -0002 2>/dev/null | wc -l | tr -d ' ')
    wwd=$(eval find / $FIND_PRUNE -type d -perm -0002 2>/dev/null | wc -l | tr -d ' ')
    [ "$wwf" -gt "$maxf" ] && score=$((score-15))
    [ "$wwd" -gt "$maxd" ] && score=$((score-15))

    local ssh_note="" w_prl="" w_pa="" c_prl="" c_pa=""
    if ssh_installed; then
        w_prl=$(ini_get SSH PermitRootLogin "$POLICY_FILE"); [ -z "$w_prl" ] && w_prl=no
        w_pa=$(ini_get SSH PasswordAuthentication "$POLICY_FILE"); [ -z "$w_pa" ] && w_pa=no
        c_prl=$(sshd_get PermitRootLogin); [ -z "$c_prl" ] && c_prl="(not set)"
        c_pa=$(sshd_get PasswordAuthentication); [ -z "$c_pa" ] && c_pa="(not set)"
        [ "$(echo "$c_prl" | tr A-Z a-z)" != "$(echo "$w_prl" | tr A-Z a-z)" ] && score=$((score-20))
        [ "$(echo "$c_pa" | tr A-Z a-z)" != "$(echo "$w_pa" | tr A-Z a-z)" ] && score=$((score-20))
    else
        ssh_note="SSH NOT installed → SSH checks are N/A (no scoring penalty)"
    fi
}

# ----- Help -----
help() {
    cat <<EOF
    Help      Write Out  Where Is    Cut         Execute
Exit        Read File  Replace     Paste       Justify
Location    Go To Line  Undo       Set Mark    To Bracket
Redo        Copy       Where Was
EOF
}

```

Figure 2.10: initial 35

```

GNU nano 8.6
fi
[ "$score" -lt 0 ] && score=0

{
    echo "Project: SentinelShield v1.1"
    echo "Purpose: Linux Security Audit & Hardening Framework"
    echo "Approach: Detect → Validate → Harden → Verify → Report"
    echo "Generated: $(date)"
    echo "Host: $(hostname)"
    echo "Policy: $POLICY_FILE"
    echo "Log: $LOGFILE"
    echo "-----"

    echo "[1] Permissions (REAL FS only)"
    echo "World-writable files: $wwf (max $maxf)"
    echo "World-writable dirs : $wwd (max $maxd)"
    echo

    echo "[2] SSH Security"
    if ssh_installed; then
        echo "SSH installed: YES | Service: $(ssh_status)"
        echo "Rule: Unset SSH options are NON-COMPLIANT (FAIL)"
        echo "PermitRootLogin      : Current=$c_prl | Policy=$w_prl"
        echo "PasswordAuthentication : Current=$c_pa  | Policy=$w_pa"
    else
        echo "SSH installed: NO"
        echo "$ssh_note"
        echo "Install:"
        echo "  sudo apt install openssh-server"
        echo "  sudo systemctl enable --now ssh"
    fi
    echo

    echo "[3] Ports (top 60)"
    if have ss; then ss -tulpen 2>/dev/null | head -n 60
    elif have netstat; then netstat -tulpen 2>/dev/null | head -n 60
    else echo "No ss/netstat."; fi
    echo

    echo "[4] SUID/SGID (top 40)"
    eval find / $FIND_PRUNE -type f \( -perm -4000 -o -perm -2000 \) 2>/dev/null | head -n 40
    echo

    echo "[5] Services (top 25)"
    systemctl list-units --type=service --state=running 2>/dev/null | head -n 25 || true
    echo
}

Help      Write Out  Where Is  Cut       Execute   Location  M-U Undo
Exit      Read File   Replace   Paste     Justify   Go To Line M-B Redo

```

Figure 2.11: initial 36

```

echo "=== FINAL SCORE: $score / 100 ==="
} >"$rep"

log "Report $rep score=$score"
msg "Report Generated" "Saved:\n$rep\n\nLog:\n$LOGFILE"
}

view_last_report() {
    local last
    last=$(ls -l "$BACKUP_DIR"/security_report*.txt 2>/dev/null | sort -r | head -n 1 || true)
    [ -z "$last" ] && { msg "Reports" "No report found in:\n$BACKUP_DIR"; return; }
    show "Last Report: $(basename "$last")" "$last"
}

view_log() {
    [ -f "$LOGFILE" ] || { msg "Logs" "Log file not found yet."; return; }
    show "Sentinel Log" "$LOGFILE"
}

reports_menu() {
    while true; do
        local c
        c=$(whiptail --title "Reports & Logs" --menu "$(status_line)\n\nSelect:" 20 86 10 \
            "1" "Generate Report + Score" \
            "2" "View Last Report" \
            "3" "View Log File" \
            "4" "Back" 3>01 1>02 2>03) || return
        case "$c" in
            1) report_generate ;;
            2) view_last_report ;;
            3) view_log ;;
            4) return ;;
        esac
    done
}

# ----- CPU Scheduling + renice -----
cpu_overview() {
    bigmsg "CPU Module (OS Lab)" \
    "Includes OS scheduling concepts:
    - FCFS
    - SJF (Non-preemptive)
    - Round Robin (Quantum)

    Also includes renice demo (nice value changes)."
}

```

Figure 2.12: initial 37

```

GNU nano 8.6 test.sh
cpu_renice_demo() {
    local T1="$TMPDIR/sentinel_cpu_before.txt" T2="$TMPDIR/sentinel_cpu_after.txt"
    ps -eo pid,pcpu,ni,comm --sort=-pcpu | head -n 15 >"$T1"
    show "renice Demo - Before" "$T1"

    local PID
    PID=$(whiptail --inputbox "Enter PID to increase nice (+5):" 10 70 3>61 1>62 2>63) || return
    [ -z "$PID" ] && return

    {
        echo "=== renice Demo ==="
        echo "Command: renice +5 $PID"
        echo
        renice +5 "$PID"
        echo
        ps -p "$PID" -o pid,pcpu,ni,comm
    } >"$T2" 2>&1

    log "renice demo PID=$PID"
    show "renice Demo - After" "$T2"
}

PIDS=(); ARR=(); BUR=()
sched_get() {
    local N
    N=$(whiptail --inputbox "Processes count (1..12):" 10 60 "4" 3>61 1>62 2>63) || return 1
    [[ "$N" =~ ^[0-9]+$ ]] && [ "$N" -ge 1 ] && [ "$N" -le 12 ] || { msg "Input" "Invalid N"; return 1; }
    PIDS=(); ARR=(); BUR=()
    local i=0
    for ((i=1;i<=N;i++)); do
        a=$(whiptail --inputbox "Arrival time for P$i (>0):" 10 60 "0" 3>61 1>62 2>63) || return 1
        b=$(whiptail --inputbox "Burst time for P$i (>1):" 10 60 "5" 3>61 1>62 2>63) || return 1
        [[ "$a" =~ ^[0-9]+$ ]] && [[ "$b" =~ ^[0-9]+$ ]] && [ "$b" -ge 1 ] || { msg "Input" "Bad input"; return 1; }
        PIDS+=("P$i"); ARR+=("$a"); BUR+=("$b")
    done
}
sort_arrival() { local n=${#PIDS[@]} i j t; IDX=(); for ((i=0;i<n;i++)); do IDX+=("$i"); done
    for ((i=0;i<n;i++)); do for ((j=i+1;j<n;j++)); do
        [ "${ARR[${IDX[i]}]}" -gt "${ARR[${IDX[j]}]}" ] && { t=${IDX[j]}; IDX[j]=${IDX[i]}; IDX[i]=t; }
    done; done
}
sched_out() {
    local title="$1" T="$TMPDIR/sched_result.txt"
    >"$T"
    echo "=== CPU Scheduling: $title ===" >"$T"
    echo >>"$T"
}

echo "Gantt:" >>"$T"
echo "$GANTT" >>"$T"
echo >>"$T"
printf "%-6s %-5s %-5s %-5s %-5s\n" "PID" "AT" "BT" "WT" "TAT" >>"$T"
echo "-----" >>"$T"
local n=${#PIDS[@]} sum=0 sumt=0 i
for ((i=0;i<n;i++)); do
    printf "%-6s %-5s %-5s %-5s %-5s\n" "${PIDS[i]}" "${ARR[i]}" "${BUR[i]}" "${WT[i]}" "${TAT[i]}" >>"$T"
    sum=$((sum+WT[i])); sumt=$((sumt+TAT[i]))
done
echo >>"$T"
echo "Avg WT = $(awk 'BEGIN{printf "%.2f", $sum/$n}')" >>"$T"
echo "Avg TAT = $(awk 'BEGIN{printf "%.2f", $sumt/$n}')" >>"$T"
log "Scheduling $title"
show "Scheduling Result" "$T"

fcfs() {
    local n=${#PIDS[@]} i t=0; WT=(); TAT=()
    for ((i=0;i<n;i++)); do WT[i]=0; TAT[i]=0; done
    sort_arrival; GANTT="(time)"
    for idx in "${IDX[@]}; do
        [ "$t" -lt "${ARR[idx]}" ] && t=${ARR[idx]}
        WT[idx]=$((t-ARR[idx])); t=$((t+BUR[idx])); TAT[idx]=$((t-ARR[idx]))
        GANTT="$GANTT | ${PIDS[idx]} ($t)"
    done
    sched_out "FCFS"
}

sjf_np() {
    local n=${#PIDS[@]} i; WT=(); TAT=(); DONE=()
    for ((i=0;i<n;i++)); do WT[i]=0; TAT[i]=0; DONE[i]=0; done
    sort_arrival; local t=${ARR[${IDX[0]}]}; done=0; GANTT="(time)"
    while [ "$done" -lt "$n" ]; do
        local pick=-1 best=999999
        for ((i=0;i<n;i++)); do
            [ "${DONE[i]}" -eq 0 ] && [ "${ARR[i]}" -le "$t" ] && [ "${BUR[i]}" -lt "$best" ] && { pick=$i; best=${BUR[i]}; }
        done
        if [ "$pick" -eq -1 ]; then
            local next=999999
            for ((i=0;i<n;i++)); do [ "${DONE[i]}" -eq 0 ] && [ "${ARR[i]}" -lt "$next" ] && next=${ARR[i]}; done
            t=$next; continue
        fi
        WT[pick]=$((t-ARR[pick])); t=$((t+BUR[pick])); TAT[pick]=$((t-ARR[pick]))
        DONE[pick]=1; done=$((done+1)); GANTT="$GANTT | ${PIDS[pick]} ($t)"
    done
    sched_out "SJF (Non-preemptive)"
}

```

Figure 2.13: initial 38

```

echo "Gantt:" >>"$T"
echo "$GANTT" >>"$T"
echo >>"$T"
printf "%-6s %-5s %-5s %-5s %-5s\n" "PID" "AT" "BT" "WT" "TAT" >>"$T"
echo "-----" >>"$T"
local n=${#PIDS[@]} sum=0 sumt=0 i
for ((i=0;i<n;i++)); do
    printf "%-6s %-5s %-5s %-5s %-5s\n" "${PIDS[i]}" "${ARR[i]}" "${BUR[i]}" "${WT[i]}" "${TAT[i]}" >>"$T"
    sum=$((sum+WT[i])); sumt=$((sumt+TAT[i]))
done
echo >>"$T"
echo "Avg WT = $(awk 'BEGIN{printf "%.2f", $sum/$n}')" >>"$T"
echo "Avg TAT = $(awk 'BEGIN{printf "%.2f", $sumt/$n}')" >>"$T"
log "Scheduling $title"
show "Scheduling Result" "$T"

fcfs() {
    local n=${#PIDS[@]} i t=0; WT=(); TAT=()
    for ((i=0;i<n;i++)); do WT[i]=0; TAT[i]=0; done
    sort_arrival; GANTT="(time)"
    for idx in "${IDX[@]}; do
        [ "$t" -lt "${ARR[idx]}" ] && t=${ARR[idx]}
        WT[idx]=$((t-ARR[idx])); t=$((t+BUR[idx])); TAT[idx]=$((t-ARR[idx]))
        GANTT="$GANTT | ${PIDS[idx]} ($t)"
    done
    sched_out "FCFS"
}

sjf_np() {
    local n=${#PIDS[@]} i; WT=(); TAT=(); DONE=()
    for ((i=0;i<n;i++)); do WT[i]=0; TAT[i]=0; DONE[i]=0; done
    sort_arrival; local t=${ARR[${IDX[0]}]}; done=0; GANTT="(time)"
    while [ "$done" -lt "$n" ]; do
        local pick=-1 best=999999
        for ((i=0;i<n;i++)); do
            [ "${DONE[i]}" -eq 0 ] && [ "${ARR[i]}" -le "$t" ] && [ "${BUR[i]}" -lt "$best" ] && { pick=$i; best=${BUR[i]}; }
        done
        if [ "$pick" -eq -1 ]; then
            local next=999999
            for ((i=0;i<n;i++)); do [ "${DONE[i]}" -eq 0 ] && [ "${ARR[i]}" -lt "$next" ] && next=${ARR[i]}; done
            t=$next; continue
        fi
        WT[pick]=$((t-ARR[pick])); t=$((t+BUR[pick])); TAT[pick]=$((t-ARR[pick]))
        DONE[pick]=1; done=$((done+1)); GANTT="$GANTT | ${PIDS[pick]} ($t)"
    done
    sched_out "SJF (Non-preemptive)"
}

```

Figure 2.14: insert 39

```

Session Actions Edit View Help
GNU nano 8.6 test.sh

c) {
local q; q=$(whiptail --inputbox "Round Robin quantum (>1):" 10 60 "2" 3>61 1>62 2>63) || return
[[ "$q" =~ ^[0-9]+$ ]] && [ "$q" -ge 1 ] || { msg "Input" "Invalid quantum"; return; }
local n=${#PIDS[@]}; i; WT=(); TAT=(); REM=(); FIN=(); ENQ=()
for ((i=0;i<n;i++)); do WT[i]=0; TAT[i]=0; REM[i]=${ENQ[i]}; FIN[i]=0; ENQ[i]=0; done
sort arrival; local t=${ARR[${IDX[0]}]}; queue=(); head=0; done=0; GANTT="(time)"
for ((i=0;i<n;i++)); do [ "${ARR[i]}" -le "$t" ] && [ "${ENQ[i]}" -eq 0 ] && { queue+=("$i"); ENQ[i]=1; }; done
while [ "$done" -lt "$n" ]; do
if [ "$head" -ge "${queue[0]}" ]; then
local next=999999
for ((i=0;i<n;i++)); do [ "${FIN[i]}" -eq 0 ] && [ "${ENQ[i]}" -eq 0 ] && [ "${ARR[i]}" -lt "$next" ] && next=${ARR[i]}; done
t=$next
for ((i=0;i<n;i++)); do [ "${FIN[i]}" -eq 0 ] && [ "${ENQ[i]}" -eq 0 ] && [ "${ARR[i]}" -le "$t" ] && { queue+=("$i"); ENQ[i]=1; }; done
head=0; continue
fi
local p=${queue[head]}; head=$((head+1))
local run=${REM[p]} -lt "$run" ] && run=${REM[p]}
t=$((t+run)); REM[p]=${REM[p]-run}; GANTT="$GANTT | ${PIDS[p]} (+$run=>$t)"
for ((i=0;i<n;i++)); do [ "${FIN[i]}" -eq 0 ] && [ "${ENQ[i]}" -eq 0 ] && [ "${ARR[i]}" -le "$t" ] && { queue+=("$i"); ENQ[i]=1; }; done
if [ "${REM[p]}" -eq 0 ]; then FIN[p]=1; done=$((done+1)); TAT[p]=${t-ARR[p]}; else queue+=("$p"); fi
[ "$head" -gt 25 ] && { queue=("${queue[@]:head}"); head=0; }
done
for ((i=0;i<n;i++)); do WT[i]=${TAT[i]-BUR[i]}; done
sched_out "Round Robin (q=$q)"

cpu_alg_menu() {
sched_get || return
while true; do
local c
c=$(whiptail --title "CPU Scheduling" --menu "Select algorithm:" 18 70 10 \
"1" "FCFS" \
"2" "SJF (Non-preemptive)" \
"3" "Round Robin" \
"4" "Back" 3>61 1>62 2>63) || return
case "$c" in
1) fcfs ;;
2) sjf_np ;;
3) rr ;;
4) return ;;
esac
done

cpu_menu() {
while true; do
local c
c=$(whiptail --title "CPU Module (OS Lab)" --menu "${status_line}\n\nSelect:" 20 86 10 \
"1" "Overview" \

```

Figure 2.15: initial 40

```

"2" "Algorithms (FCFS/SJF/RR)" \
"3" "renice Demo" \
"4" "Back" 3>61 1>62 2>63) || return
case "$c" in
1) cpu_overview ;;
2) cpu_alg_menu ;;
3) cpu_renice_demo ;;
4) return ;;
esac
done

# ----- Main -----
main_menu() {
need whiptail
ensure_policy
ensure
intro
env_check

while true; do
local c
c=$(whiptail --title "SentinelShield - Main Menu" --menu "${status_line}\n\nChoose a module:" 20 86 10 \
"1" "Security Audit" \
"2" "SSH Hardening (Policy + Apply + Rollback)" \
"3" "Reports & Logs" \
"4" "CPU Scheduling (OS Lab)" \
"5" "Exit" 3>61 1>62 2>63)

[ $? -ne 0 ] && clear && exit 0
case "$c" in
1) audit_menu ;;
2) hardening_menu ;;
3) reports_menu ;;
4) cpu_menu ;;
5) clear; exit 0 ;;
esac
done
}

main_menu

```

Figure 2.16: initial 41

2. First, the system performs a security audit by scanning key indicators such as file permissions, world-writable files and directories, open ports, SUID/SGID binaries, and running services. This step identifies potential security risks present in the system.

3. Next, the framework executes policy validation, where the current system configuration is compared against predefined security rules stored in an INI-based policy file. Each rule is evaluated and marked as PASS or FAIL, ensuring transparent compliance checking.

4. If applicable, the system then proceeds to secure hardening, specifically targeting SSH configuration. Before applying any changes, SentinelShield creates a backup of the existing configuration. The new settings are applied according to policy rules, validated using configuration checks, and safely restarted. In case of failure, an automatic rollback mechanism restores the previous configuration.

5. Finally, SentinelShield generates a security report and score, summarizing audit findings, policy compliance, applied changes, and overall system security status. All activities are logged to maintain traceability and provide evidence for analysis.

Tools and libraries

Bash Script: Core language used to implement system-level automation and logic.

Whiptail: Provides a menu-driven, text-based user interface in the terminal.

find: Used to scan file permissions and detect world-writable files and directories.

ss / netstat: Used to identify open and listening network ports.

systemctl: Manages and inspects system services and their states.

ps: Displays running processes for monitoring and renice demonstration.

OpenSSH (sshd): Used for SSH configuration validation and secure hardening.

renice: Demonstrates real-time process priority adjustment in Linux.

awk: Parses policy (INI) files and extracts configuration values.

2.3 Algorithms

The algorithms and the programming codes in detail should be included .

Pseudo-codes are also encouraged very much to be included in this chapter for your project.

- Bullet points can also be included anywhere in this project report.

Algorithm 1: Sample Algorithm

- 1 The script starts by initializing the backup directory, log file, and security policy file. If the policy file does not exist, a default policy is automatically created.
 - 2 A menu-driven user interface is displayed using Whiptail, allowing the user to select different operations.
 - 3 When a security audit option is selected, the system scans world-writable files and directories, checks sticky-bit permissions, lists open network ports, identifies SUID and SGID binaries, and displays running and enabled services.
 - 4 During policy validation, the framework reads expected values from the policy file and compares them with the current system configuration, marking each rule as PASS or FAIL while treating unset values as non-compliant.
 - 5 For SSH hardening, the script verifies root privileges and the availability of SSH, creates a timestamped backup of the configuration file, applies policy-defined settings, validates the configuration, and restarts the SSH service.
 - 6 If configuration validation or service restart fails, the previous configuration is automatically restored using the backup mechanism.
 - 7 A manual rollback option allows the user to restore any previously saved SSH configuration backup.
 - 8 The reporting module collects audit and policy results, calculates a security score based on detected risks, generates a detailed report, and records all activities in a log file.
 - 9 The CPU scheduling module simulates FCFS, SJF (non-preemptive), and Round Robin algorithms by calculating waiting time and turnaround time, and also demonstrates real process priority control using the renice command.
 - 10 The system returns to the main menu after each operation and terminates gracefully when the user chooses to exit.
-

Chapter 3

Performance Evaluation

3.1 Simulation Environment/ Simulation Procedure

Simulation Environment :

Operating System: Ubuntu Linux

Shell: Bash (default Ubuntu shell environment)

Interface: Terminal (UI based)

Privileges: Normal user for audit/report viewing, sudo/root required for SSH hardening, rollback, and renice demo

Required Packages/Tools: whiptail, openssh-server.

Simulation Procedure:

Prepare the script: Save the project as SentinelShieldFinal.sh and make it executable using `chmod +x SentinelShieldFinal.sh`.

Run the framework: Start using `sudo ./SentinelShieldFinal.sh` for full feature access (audit-only can run without sudo).

Audit testing: From the menu, run Basic Scan, Ports Scan, SUID/SGID Scan, and Services Overview to collect security evidence.

Policy testing: Open the Policy Validation option to verify PASS/FAIL results against the INI policy file.

Hardening testing: Apply SSH hardening (backup → apply → validate → restart) and verify that rollback occurs if validation/restart fails.

Report generation: Generate the final security report and confirm that a score and log entries are created in the backup directory.

CPU simulation testing: Run FCFS/SJF/RR simulations and verify Gantt chart, WT, and TAT outputs; run renice demo to observe real priority changes.

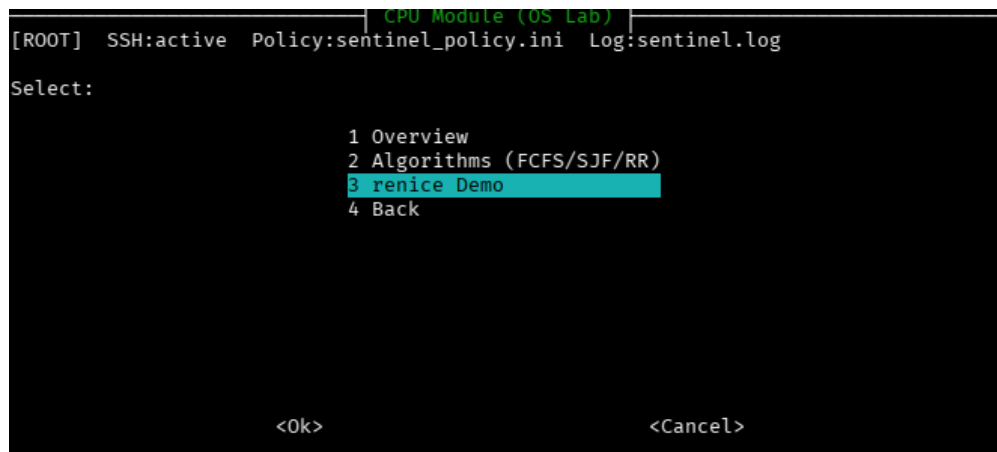


Figure 3.1: initial 23

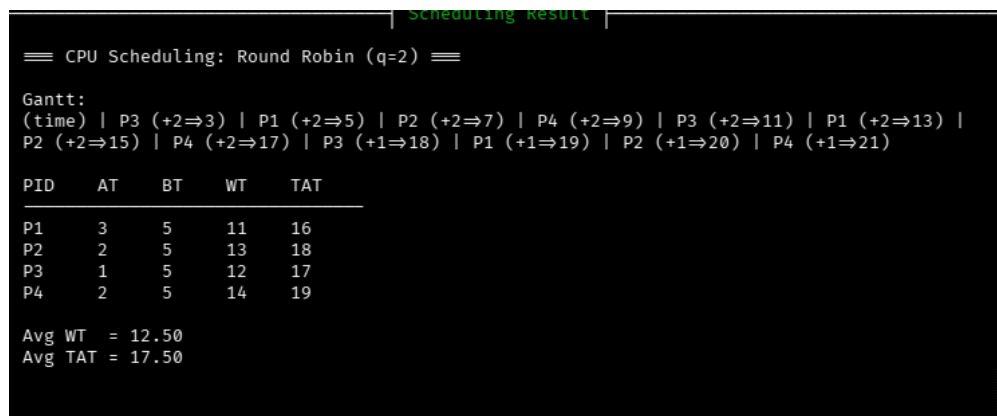


Figure 3.2: initial 22



Figure 3.3: initial 21

```
SentinelShield - Main Menu
[ROOT] SSH:active Policy:sentinel_policy.ini Log:sentinel.log

Choose a module:

1 Security Audit
2 SSH Hardening (Policy + Apply + Rollback)
3 Reports & Logs
4 CPU Scheduling (OS Lab)
5 Exit

<Ok> <Cancel>
```

Figure 3.4: initial 20

```
Last Report: security_report_2025-12-25_05-35-15.txt

Project: SentinelShield v1.1
Purpose: Linux Security Audit & Hardening Framework
Approach: Detect → Validate → Harden → Verify → Report
Generated: Thu Dec 25 05:35:15 AM EST 2025
Host: kali
Policy: /root/sentinel_backup/sentinel_policy.ini
Log: /root/sentinel_backup/sentinel.log

[1] Permissions (REAL FS only)
World-writable files: 0 (max 50)
World-writable dirs : 0 (max 50)

[2] SSH Security
SSH installed: YES | Service: active
Rule: Unset SSH options are NON-COMPLIANT (FAIL)
PermitRootLogin : Current=(not set) | Policy=no
PasswordAuthentication : Current=(not set) | Policy=no

<Ok>
```

Figure 3.5: initial 19

3.2 Results Analysis/Testing

The testing of SentinelShield was conducted by executing each module on a Linux system and observing the generated outputs, which are included in the attached screenshots. The security audit module successfully identified permission configurations, validated sticky-bit settings, listed active services, and displayed open network ports without producing false positives from virtual file systems. Policy validation accurately reflected PASS and FAIL states based on the defined security rules, and SSH hardening correctly handled both applicable and non-applicable scenarios depending on service availability. The reporting module generated consistent security scores aligned with detected risks, while the CPU scheduling module produced correct scheduling sequences, Gantt charts, and timing calculations, confirming the accuracy of the implemented algorithms. Overall, the results demonstrate that the framework operates reliably and produces expected, verifiable outputs across all modules..

```
Sentinel Log
[2025-12-24 15:14:35] Created policy /root/sentinel_backup/sentinel_policy.ini
[2025-12-25 05:26:29] Audit: basic scan
[2025-12-25 05:27:03] Audit: ports
[2025-12-25 05:28:11] Audit: suid/sgid
[2025-12-25 05:29:16] Audit: services
[2025-12-25 05:31:03] Hardening: policy check
[2025-12-25 05:31:34] Hardening applied.
Backup=/root/sentinel_backup/sshd_config.2025-12-25_05-31-33.bak
[2025-12-25 05:32:26] Rollback restored
/root/sentinel_backup/sshd_config.2025-12-25_05-31-33.bak
[2025-12-25 05:35:15] Report /root/sentinel_backup/security_report_2025-12-25_05-35-15.txt
score=60

<Ok>
```

Figure 3.6: initial 18

```
Report Generated

Saved:
/root/sentinel_backup/security_report_2025-12-25_05-35-15.txt

Log:
/root/sentinel_backup/sentinel.log

<Ok>
```

Figure 3.7: initial 17

```
Reports & Logs
[ROOT] SSH:active Policy:sentinel_policy.ini Log:sentinel.log

Select:

1 Generate Report + Score
2 View Last Report
3 View Log File
4 Back

<Ok> <Cancel>
```

Figure 3.8: initial 16

```
SentinelShield v1.1
Advanced Linux Security Hardening & Threat Audit Framework

Purpose: Detect → Validate → Harden → Verify → Report

Modules:
• Security Audit (permissions, ports, suid/sgid, services)
• Policy-based PASS/FAIL (INI policy)
• Safe SSH Hardening (backup + validate + restart + rollback)
• Security Report + Score + Logs
• CPU Scheduling (FCFS/SJF/RR) + renice demo (OS Lab)

[USER] SSH:active Policy:sentinel_policy.ini Log:sentinel.log

<Ok>
```

Figure 3.9: Initial

```
Environment Check

== Environment Check ==
Generated: Thu Dec 25 05:24:18 AM EST 2025

Core:
- whiptail: OK

Audit tools:
- ss: OK
- netstat: OK

SSH hardening tools:
- OpenSSH Server: INSTALLED
- sshd_config: /etc/ssh/sshd_config
- service: active

Tip: Run with sudo for full coverage.

<Ok>
```

Figure 3.10: initial 2

```
SentinelShield - Main Menu

[ROOT] SSH:active Policy:sentinel_policy.ini Log:sentinel.log

Choose a module:

1 Security Audit
2 SSH Hardening (Policy + Apply + Rollback)
3 Reports & Logs
4 CPU Scheduling (OS Lab)
5 Exit

<Ok> <Cancel>
```

Figure 3.11: initial3

```
[ROOT] SSH:active Policy:sentinel_policy.ini Log:sentinel.log
Select:
1 Basic Scan (permissions + sticky-bit)
2 Open Ports Scan
3 SUID/SGID Scan
4 Services Overview
5 Back

<Ok> <Cancel>
```

Figure 3.12: initial 4

```
== Basic Security Scan ==
Generated: Thu Dec 25 05:22:13 AM EST 2025

[USER] SSH:active Policy:sentinel_policy.ini Log:sentinel.log

[+] World-writable files (top 10, REAL FS only):
None found.

[+] World-writable directories (top 10, REAL FS only):
None found.

[+] Sticky-bit check (expected 1777 with 't'):
drwxrwxrwt 1777 root:root /tmp
drwxrwxrwt 1777 root:root /var/tmp
drwxrwxrwt 1777 root:root /dev/shm
drwxrwxrwt 1777 root:root /run/lock
drwxrwxrwt 1777 root:utmp /run/screen

<Ok>
```

Figure 3.13: initial 5

```
== Open / Listening Ports ==
Generated: Thu Dec 25 05:27:03 AM EST 2025

[ROOT] SSH:active Policy:sentinel_policy.ini Log:sentinel.log

Netid State Recv-Q Send-Q Local Address:Port Peer Address:PortProcess
tcp LISTEN 0 128 0.0.0.0:22 0.0.0.0:*
users:(("sshd",pid=4592,fd=6)) ino:18019 sk:1 cgroup:/system.slice/ssh.service ↔
tcp LISTEN 0 128 [::]:22 [::]:*
users:(("sshd",pid=4592,fd=7)) ino:18021 sk:2 cgroup:/system.slice/ssh.service v6only:1
↔

<Ok>
```

Figure 3.14: initial 6

```

Audit - SUID/SGID
===== SUID/SGID Binaries (REAL FS only) =====
Generated: Thu Dec 25 05:28:11 AM EST 2025

[ROOT] SSH:active Policy:sentinel_policy.ini Log:sentinel.log

None found (or restricted permissions).

<Ok>

```

Figure 3.15: initial 7

```

Audit - Services
===== Services Overview =====
Generated: Thu Dec 25 05:29:15 AM EST 2025

[ROOT] SSH:active Policy:sentinel_policy.ini Log:sentinel.log

[+] Running services (top 50):
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
accounts-daemon.service            loaded active running Accounts Service
colord.service                      loaded active running Manage, Install and Generate Color
Profiles
cron.service                      loaded active running Regular background program
processing daemon
dbus.service                      loaded active running D-Bus System Message Bus
getty@tty1.service                loaded active running Getty on tty1
haveged.service                   loaded active running Entropy Daemon based on the HAVEGE
algorithm
lightdm.service                   loaded active running Light Display Manager
ModemManager.service              loaded active running Modem Manager

<Ok>

```

Figure 3.16: initial 8

```

SentinelShield - Main Menu
[ROOT] SSH:active Policy:sentinel_policy.ini Log:sentinel.log

Choose a module:

1 Security Audit
2 SSH Hardening (Policy + Apply + Rollback)
3 Reports & Logs
4 CPU Scheduling (OS Lab)
5 Exit

<Ok>                                <Cancel>

```

Figure 3.17: initial 9

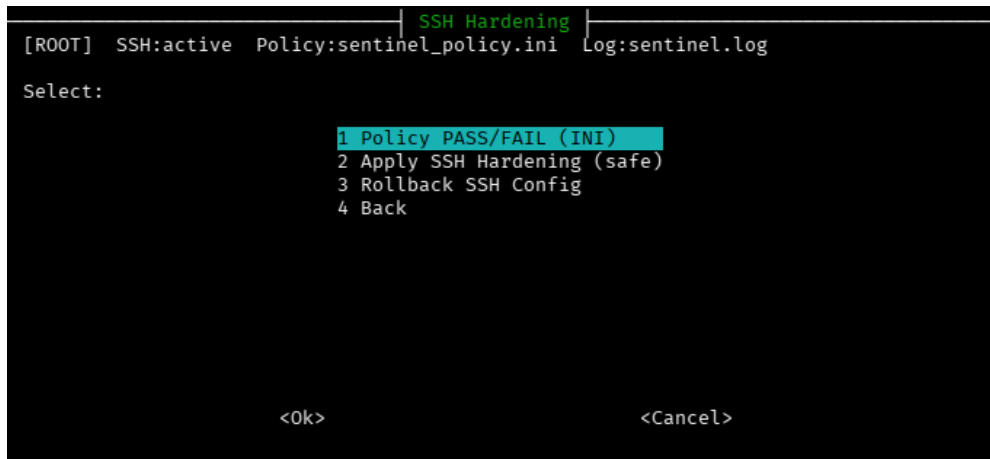


Figure 3.18: initial 10



Figure 3.19: initial 25

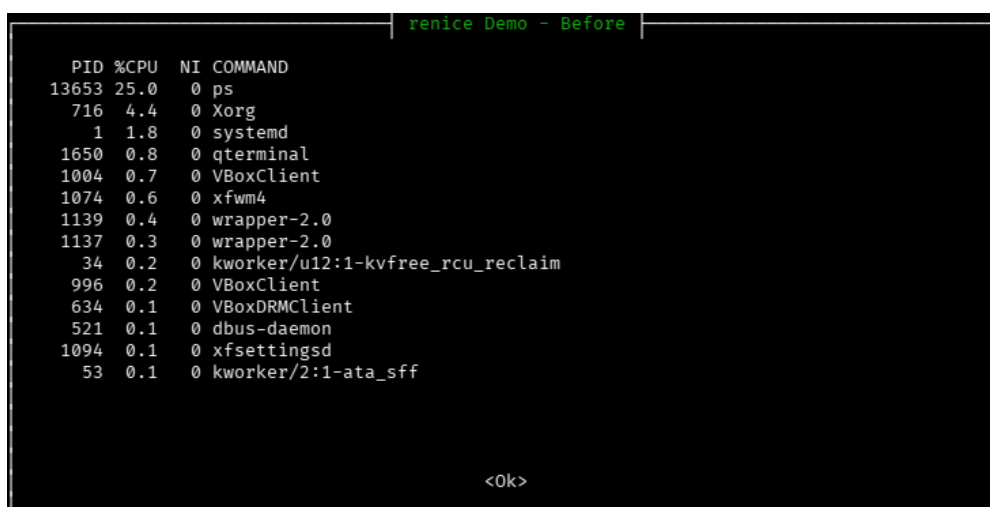


Figure 3.20: initial 24

```
Policy Check

== Policy Validation (INI) ==
Policy file: /root/sentinel_backup/sentinel_policy.ini
Generated: Thu Dec 25 05:31:03 AM EST 2025

[ROOT] SSH:active Policy:sentinel_policy.ini Log:sentinel.log

Rule: Unset SSH options are NON-COMPLIANT (FAIL).

[+] PermitRootLogin
  Current: (not set)
  Policy : no
  => FAIL

[+] PasswordAuthentication
  Current: (not set)
  Policy : no
  => FAIL

<Ok>
```

Figure 3.21: initial 11

```
Hardening

Hardening applied successfully.
Backup:
/root/sentinel_backup/sshd_config.2025-12-25_05-31-33.bak

<Ok>
```

Figure 3.22: initial 12

```
Rollback

1 sshd_config.2025-12-25_05-31-33.bak
```

Figure 3.23: initial 13

```
Rollback

Restore this backup?
sshd_config.2025-12-25_05-31-33.bak

<Yes> <No>
```

Figure 3.24: initial 14

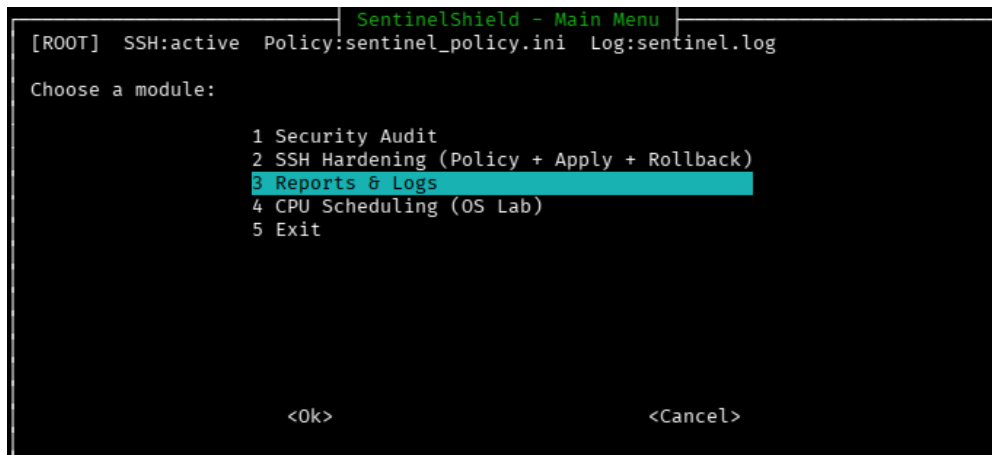


Figure 3.25: initial 15

3.3 Results Overall Discussion

The results obtained from the SentinelShield project demonstrate that the framework successfully achieves its intended objectives of Linux security auditing and safe hardening. The system was able to accurately identify configuration-level security issues such as file permission risks, open network ports, active services, and SUID/SGID binaries while excluding non-relevant virtual file systems to maintain result accuracy. Policy validation produced clear PASS and FAIL outcomes based on predefined rules, ensuring consistent security assessment across different system states. The SSH hardening module operated safely by creating backups, validating configurations before restart, and supporting rollback, which confirmed the reliability of the safety mechanism. Additionally, the CPU scheduling simulations correctly generated Gantt charts, waiting time, and turnaround time values, reinforcing operating system concepts through practical demonstration. Overall, the project outputs indicate that SentinelShield functions as a stable, structured, and educational security framework suitable for both academic and real-world configuration auditing scenarios.

3.3.1 Complex Engineering Problem Discussion

The development of SentinelShield addresses a complex engineering problem involving the secure auditing and controlled hardening of Linux operating systems. Linux security is inherently complex due to the interaction between file systems, network services, system daemons, user privileges, and configuration dependencies. Applying security changes without proper validation can result in system instability or loss of remote access, particularly in SSH-based environments. The project tackles this complexity by introducing a structured, policy-driven approach that separates detection, validation, execution, and verification stages. Managing conditional execution based on system state, ensuring real file system accuracy, handling privilege requirements, and implementing reliable rollback mechanisms required careful design and error handling. Additionally, integrating security auditing with operating system concepts such as CPU scheduling simulations further increased system complexity. By modularizing these components and enforcing safety checks at each stage, SentinelShield provides a controlled and re-

peatable solution to a multifaceted security engineering problem.

Chapter 4

Conclusion

4.1 Discussion

This project presents the successful development of SentinelShield, a Bash-based Linux security audit and hardening framework that combines system assessment, policy validation, and controlled configuration management into a single, menu-driven tool. The experimental results demonstrate that the framework effectively identifies common security issues such as insecure file permissions, exposed services, and policy mismatches, while avoiding false positives by excluding virtual file systems. Policy validation outputs clearly reflect system compliance through PASS and FAIL indicators, enabling straightforward interpretation of results. The SSH hardening module proved reliable by applying secure configurations only after creating backups and validating changes, with automatic rollback ensuring system stability in failure scenarios. Additionally, the reporting and scoring mechanism provided a summarized yet informative view of the system's security posture. The CPU scheduling simulations and renice demonstration further reinforced operating system concepts by combining theoretical understanding with practical observation. Overall, the results and observations confirm that SentinelShield functions as intended, offering a dependable and educational approach to Linux system security assessment and administration.

4.2 Limitations

While SentinelShield provides a structured, policy-based framework for Linux security auditing and safe SSH hardening, it has several inherent limitations. The system focuses mainly on configuration checks, permission analysis, and service visibility, and therefore does not perform deep vulnerability assessments, exploit detection, or real-time intrusion monitoring. The CPU scheduling module is designed strictly for educational purposes, simulating scheduling algorithms rather than interacting with or modifying the kernel's actual scheduler, with the exception of a limited renice demonstration. Additionally, certain features such as SSH hardening, rollback, and comprehensive scans require root privileges and depend on the availability of specific services like OpenSSH and systemd, which may limit full functionality on minimal, containerized, or non-standard Linux environments.

4.3 Scope of Future Work

SentinelShield can be further enhanced to improve its usability, security coverage, and real-world applicability. Future versions may incorporate firewall hardening using tools such as UFW or iptables and extend policy validation to cover additional services beyond SSH, including web and database services. Integration with recognized security standards like CIS benchmarks could provide more comprehensive compliance checking. Automated scheduling of periodic scans with alert notifications, as well as exporting reports in formats such as HTML or PDF, would improve continuous monitoring and reporting. Additionally, enhancing modularity and adding support for containerized or cloud-based Linux environments could broaden the framework's deployment scope.

References

- [1] Omid C Farokhzad and Robert Langer. Impact of nanotechnology on drug delivery. *ACS nano*, 3(1):16–20, 2009.
- [2] Uthayasankar Sivarajah, Muhammad Mustafa Kamal, Zahir Irani, and Vishanth Weerakkody. Critical analysis of big data challenges and analytical methods. *Journal of Business Research*, 70:263–286, 2017.
- [3] Douglas Laney. 3d data management: controlling data volume, velocity and variety. gartner, 2001.
- [4] MS Windows NT kernel description. <http://web.archive.org/web/20080207010024/http://www.808multimedia.com/winnt/kernel.htm>. Accessed Date: 2010-09-30.